

Loan Status Prediction Model

Loan Status Prediction Model

📄 loan_data.csv

📄 Loan_Status_Prediction_Model.ipynb

1. Importing Required Libraries

```
import pandas as pd
```

- The pandas library is imported.
- It is used for data handling, reading CSV files, and working with tabular data using DataFrames.

2. Loading the Dataset

```
df = pd.read_csv("/content/loan_data.csv")
```

- The loan dataset is loaded from a CSV file.
- The data is stored in a DataFrame named df.
- This DataFrame will be used for all further operations.

3. Viewing the Dataset

```
df.head()
```

- Displays the first five rows of the dataset.
- Helps to understand the structure of the data, column names, and sample values.

4. Checking for Missing Values

```
df.isna().sum()
```

- Checks each column for missing (NaN) values.
- Returns the count of missing values column-wise.
- This step is important to verify data completeness.

5. Total Number of Records

```
len(df)
```

- Returns the total number of rows (entries) in the dataset.
- Helps to understand the dataset size.

6. Encoding Categorical Data

Machine learning models require numerical values. The code converts categorical (string) columns into numeric values using **Label Encoding**.

6.1 Encoding person_gender

```
from sklearn.preprocessing import LabelEncoder  
le = LabelEncoder()  
df['person_gender'] = le.fit_transform(df['person_gender'])
```

- Converts gender values into numeric form.
- Example: Male → 1, Female → 0 (actual mapping depends on data).

6.2 Encoding person_education

```
le = LabelEncoder()  
df['person_education'] = le.fit_transform(df['person_education'])
```

- Converts education categories into numbers.
- Each education level gets a unique numeric value.

6.3 Encoding person_home_ownership

```
le = LabelEncoder()  
df['person_home_ownership'] = le.fit_transform(df['person_home_ownership'])
```

- Converts home ownership status into numeric labels.

6.4 Encoding loan_intent

```
le = LabelEncoder()  
df['loan_intent'] = le.fit_transform(df['loan_intent'])
```

- Converts the purpose of the loan into numeric form.

6.5 Encoding previous_loan_defaults_on_file

```
le = LabelEncoder()  
df['previous_loan_defaults_on_file'] =  
le.fit_transform(df['previous_loan_defaults_on_file'])
```

- Converts previous loan default history into numeric values.
- Typically represents Yes/No information.

6.6 Viewing Encoded Classes

```
le.classes_
```

- Displays the original class labels before encoding.

- Helps to understand how categorical values were converted.

7. Preparing Data for Training

7.1 Separating Features and Target

```
x = df.drop('loan_status', axis=1)
y = df['loan_status']
```

- x contains all input features.
- y contains the target variable (loan_status).

7.2 Splitting Training and Testing Data

```
from sklearn.model_selection import train_test_split
```

- The dataset is split into training and testing sets.
- Training data is used to train the model.
- Testing data is used to evaluate model performance.

8. Model Training Using Logistic Regression

8.1 Initial Model Training

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(x_train, y_train)
```

- A Logistic Regression model is created.
- The model is trained using training data.

8.2 Model Training with Increased Iterations

```
lr = LogisticRegression(max_iter=1000)
lr.fit(x_train, y_train)
```

- The model is retrained with max_iter = 1000.
- Increasing iterations helps the model converge properly.

9. Model Prediction

```
y_pred = lr.predict(x_test)
```

- The trained model predicts loan status for test data.
- Predicted values are stored in y_pred.

10. Comparing Actual and Predicted Values

y_test
y_pred

- y_test contains actual loan status values.
- y_pred contains predicted loan status values.

11. Model Evaluation

11.1 Classification Report

```
from sklearn.metrics import classification_report  
print(classification_report(y_test, y_pred))
```

- Evaluates model performance.
- Displays:
 - Precision
 - Recall
 - F1-Score
 - Support
- Helps understand how well the model predicts loan approval or rejection.

Conclusion

This code builds a **Loan Status Prediction Model** using **Logistic Regression**. The process includes:

- Loading and exploring data
- Encoding categorical values
- Splitting data into training and testing sets
- Training a Logistic Regression model
- Predicting loan status
- Evaluating performance using a classification report

Updated ML Model

UPDATED ML Model

USING

- Standardizing
- Under-Sampling
- XGBoost Algorithm

🔗 Loan Status Prediction Model.ipynb

12. Train–Test Split with Stratification

```
x = df.drop('loan_status', axis=1)
y = df['loan_status']
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    x, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Explanation

- The dataset is divided into **80% training data** and **20% testing data**.
- `loan_status` is removed from `x` to prevent **data leakage**.
- `stratify=y` ensures that the proportion of approved and rejected loans remains consistent in both training and testing sets.
- `random_state=42` guarantees reproducibility of results.

13. Feature Scaling using StandardScaler (Logistic Regression Only)

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
x_train_std = scaler.fit_transform(x_train)
x_test_std = scaler.transform(x_test)
```

Explanation

- Logistic Regression is sensitive to feature scale.

- `fit_transform` is applied **only on training data** to learn mean and standard deviation.
- `transform` is applied on test data using the same statistics.
- This avoids **data leakage** and ensures consistent feature scaling.

14. Logistic Regression Model Training (Improved)

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression(
    max_iter=1000,
    class_weight="balanced"
)
```

```
lr.fit(x_train_std, y_train)
```

Explanation

- `max_iter=1000` ensures proper convergence.
- `class_weight='balanced'` compensates for class imbalance.
- The model learns a linear decision boundary between approved and rejected loans.

15. Model Prediction on Test Data

```
y_pred = lr.predict(x_test_std)
```

Explanation

- Predictions are generated for unseen test data.
- Output values:
 - 1 → Loan Approved
 - 0 → Loan Rejected

16. Model Evaluation

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

Observed Results

- Accuracy ≈ **89%**
- Strong balance between approval and rejection predictions
- Significantly improved recall for minority class compared to initial model

This confirms that preprocessing and model usage are now correct.

17. Advanced Model: XGBoost Classifier

```
from xgboost import XGBClassifier

scale = len(y_train[y_train==0]) / len(y_train[y_train==1])

xgb = XGBClassifier(
    n_estimators=300,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=scale,
    random_state=42
)

xgb.fit(x_train, y_train)
```

Explanation

- XGBoost handles non-linear feature interactions.
- No feature scaling is required for tree-based models.
- `scale_pos_weight` improves performance on imbalanced data.

18. XGBoost Prediction and Evaluation

```
y_pred_xgb = xgb.predict(x_test)
print(classification_report(y_test, y_pred_xgb))
```

Final Performance

- Accuracy $\approx$ **91%**
- Recall (Loan Approved) $\approx$ **92%**
- Balanced macro and weighted scores

This model outperforms Logistic Regression and is suitable for deployment.

19. Final Interpretation

- Logistic Regression provides an interpretable and stable baseline.
- XGBoost delivers higher predictive performance.
- All preprocessing steps are now model-appropriate.
- No data leakage or metric inflation exists.

Customized Phi3 : mini Model

Commands to do it

Install and run xterm

```
!pip install colab-xterm
```

```
%load_ext colabxterm
```

```
%xterm
```

In xterm, run following commands ollama and run in background)

```
curl https://ollama.ai/install.sh | sh
```

```
ollama serve &
```

Download and run any LLM model from Ollama website, example:

```
ollama run llama3.2:1b
```

Ollama CMD

```
apt-get update
```

```
apt-get install -y zstd
```

```
curl -fsSL https://ollama.com/install.sh | sh
```

```
ollama serve &
```

```
ollama pull phi3:mini
```

```
ollama create karthikeyan -f /content/Karthikeyan
```

```
🔗 Model.ipynb
```

Running Ollama LLMs in Google Colab Using XTerm to Create a Customized Model

Objective

Run Ollama inside Google Colab, keep it running in the background, download a base LLM, and create a **custom security-focused model** using a Modelfile.

1. Install and Launch XTerm in Google Colab

Google Colab does not provide a native interactive terminal. colab-xterm creates a browser-based terminal.

```
# commands to install and run my model
apt-get update
apt-get install -y zstd
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &
ollama pull phi3:mini
ollama create Karthikeyan -f /content/Karthikeyan
```

Step 1.1 – Install colab-xterm

`!pip install colab-xterm`

```
[1] !pip install colab-xterm
... Collecting colab-xterm
  Downloading colab_xterm-0.2.0-py3-none-any.whl.metadata (1.2 kB)
Requirement already satisfied: ptyprocess<0.7.0 in /usr/local/lib/python3.12/dist-packages (from colab-xterm) (0.7.0)
Requirement already satisfied: tornado>5.1 in /usr/local/lib/python3.12/dist-packages (from colab-xterm) (6.5.1)
  Downloading colab_xterm-0.2.0-py3-none-any.whl (115 kB)
    115.6/115.6 kB 8.9 MB/s eta 0:00:00
Installing collected packages: colab-xterm
Successfully installed colab-xterm-0.2.0
```

Purpose

- Installs an extension that provides an interactive Linux terminal inside Colab.

Step 1.2 – Load the XTerm Extension

`%load_ext colabxterm`

```
%load_ext colabxterm
%xterm

... Launching Xterm...
/content#
```

Purpose

- Registers the xterm extension with the current notebook runtime.

Step 1.3 – Launch the Terminal

`%xterm`

Result

- Opens a black terminal window rooted at /content
- All further commands are run **inside this terminal**

2. Install Ollama Runtime Dependencies

Ollama requires zstd for model decompression.

```
# commands to install and run my model
apt-get update
apt-get install -y zstd
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &
ollama pull phi3:mini
ollama create Karthikeyan -f /content/Karthikeyan
```

Step 2.1 – Update Package Index

apt-get update

```
/content# apt-get update
Get:1 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64 InRelease [1,581 B]
...
Get:2 https://cli.github.com/packages/stable InRelease [3,917 B]
Get:3 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease [3,632 B]
Get:4 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64 Packages [2,341 kB]
Get:5 https://cli.github.com/packages/stable/main amd64 Packages [356 B]
Get:6 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ Packages [83.8 kB]
Get:7 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Get:8 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [2,891 kB]
Get:9 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [9,700 kB]
Hit:10 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [62.6 kB]
Get:15 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,297 kB]
Get:16 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [6,388 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages [70.9 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [6,597 kB]
Get:19 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3,677 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,009 kB]
Get:21 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,608 kB]
Get:22 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [83.9 kB]
Get:23 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [37.2 kB]
Get:24 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease [18.1 kB]
Get:25 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu jammy InRelease [24.3 kB]
Get:26 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease [24.6 kB]
Get:27 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy/main amd64 Packages [38.8 kB]
Get:28 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu jammy/main amd64 Packages [45.0 kB]
Get:29 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy/main amd64 Packages [75.3 kB]
Fetched 39.5 MB in 9s (4,260 kB/s)
Reading package lists... Done
```

Purpose

- Refreshes package metadata from Ubuntu repositories.

Step 2.2 – Install zstd

apt-get install -y zstd

```

/content# apt-get install -y zstd
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  zstd
0 upgraded, 1 newly installed, 0 to remove and 118 not upgraded.
Need to get 603 kB of archives.
After this operation, 1,695 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 zstd amd64 1.4.8+dfsg-3build1 [603 kB]
Fetched 603 kB in 0s (3,155 kB/s)
Selecting previously unselected package zstd.
(Reading database ... 121689 files and directories currently installed.)
Preparing to unpack .../zstd_1.4.8+dfsg-3build1_amd64.deb ...
Unpacking zstd (1.4.8+dfsg-3build1) ...
Setting up zstd (1.4.8+dfsg-3build1) ...
Processing triggers for man-db (2.10.2-1) ...
/content#

```

Purpose

- Required to extract .zst model layers used by Ollama.

3. Install Ollama

Step 3.1 – Download and Install Ollama

`curl -fsSL https://ollama.com/install.sh | sh`

```

/content# curl -fsSL https://ollama.com/install.sh | sh
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
##### 100.0%
>>> Creating ollama user...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
WARNING: systemd is not running
WARNING: Unable to detect NVIDIA/AMD GPU. Install lspci or lshw to automatically detect and install GPU dependencies.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
/content#

```

What Happens Internally

- Downloads Ollama binary
- Installs it to /usr/local/bin/ollama
- Creates Ollama user and directories
- Skips systemd (not available in Colab)

4. Start Ollama Server (Background Mode)

`ollama serve &`

```

... /content# ollama serve &
[1] 2140
... /content# Couldn't find '/root/.ollama/id_ed25519'. Generating new private key.
Your new public key is:

ssh-ed25519 AAAAC3NzaC1lZDIINTE5AAAAIIGv6X4sV+pXMcmoqztNBBIfp/BEWLBOk0BF5q/hho

time=2026-02-05T11:02:20.581Z level=INFO source=routes.go:1631 msg="server config" env="map[CUDA_VISIBLE_DEVICES: GGM_L_VK_VISI
BLE_DEVICES: GPU_DEVICE_ORDINAL: HIP_VISIBLE_DEVICES: HSA_OVERRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: NO_PROXY: OLLAMA_CON
EXT_LENGTH:4096 OLLAMA_DEBUG:INFO OLLAMA_FLASH_ATTENTION:false OLLAMA_GPU_OVERHEAD:0 OLLAMA_HOST:http://127.0.0.1:11434 OLLAMA
_KEEP_ALIVE:5m0s OLLAMA_KV_CACHE_TYPE: OLLAMA_LLM_LIBRARY: OLLAMA_LOAD_TIMEOUT:5m0s OLLAMA_MAX_LOADED_MODELS:0 OLLAMA_MAX_QUEUE
:512 OLLAMA_MODELS:/root/.ollama/models OLLAMA_MULTIUSER_CACHE:false OLLAMA_NEW_ENGINE:false OLLAMA_NOHISTORY:false OLLAMA_NO
PRUNE:false OLLAMA_NUM_PARALLEL:1 OLLAMA_ORIGINS:[http://localhost https://localhost http://localhost:* https://localhost:* ht
tp://127.0.0.1 https://127.0.0.1 http://127.0.0.1:* https://127.0.0.1:* http://0.0.0.0 https://0.0.0.0 http://0.0.0.0:* https
://0.0.0.0:* app://* file://* tauri://* vscode-webview://* vscode-file://*] OLLAMA_REMOTEOS:[ollama.com] OLLAMA_SCHED_SPREAD:fal
se OLLAMA_VULKAN:false ROCR_VISIBLE_DEVICES: http proxy: https proxy: no proxy:]
time=2026-02-05T11:02:20.582Z level=INFO source=images.go:473 msg="total blobs: 0"
time=2026-02-05T11:02:20.582Z level=INFO source=images.go:480 msg="total unused blobs removed: 0"
time=2026-02-05T11:02:20.582Z level=INFO source=routes.go:1684 msg="Listening on 127.0.0.1:11434 (version 0.15.4)"
time=2026-02-05T11:02:20.584Z level=INFO source=runner.go:67 msg="discovering available GPUs..."
time=2026-02-05T11:02:20.585Z level=INFO source=runner.go:106 msg="experimental Vulkan support disabled. To enable, set OLLAMA
A_VULKAN=1"
time=2026-02-05T11:02:20.585Z level=INFO source=server.go:429 msg="starting runner" cmd="/usr/local/bin/ollama runner --ollama
--engine --port 38211"
time=2026-02-05T11:02:21.384Z level=INFO source=server.go:429 msg="starting runner" cmd="/usr/local/bin/ollama runner --ollama
--engine --port 46831"
time=2026-02-05T11:02:21.489Z level=INFO source=server.go:429 msg="starting runner" cmd="/usr/local/bin/ollama runner --ollama
--engine --port 34647"

```

Explanation

- `ollama serve` starts the local inference server
- & runs it in the background so the terminal remains usable
- Default API endpoint: `http://127.0.0.1:11434`

Verification

```
curl http://127.0.0.1:11434
```

5. Download a Base LLM Model

Example: Phi-3 Mini

```
ollama pull phi3:mini
```

Purpose

- Downloads a lightweight Microsoft Phi-3 model
- Suitable for Colab CPU environments

Result

- Model stored under:

/root/.ollama/models/

6. Run the Base Model (Optional Test)

```
ollama run phi3:mini
```

Purpose

- Opens an interactive chat session
- Confirms model and runtime are working

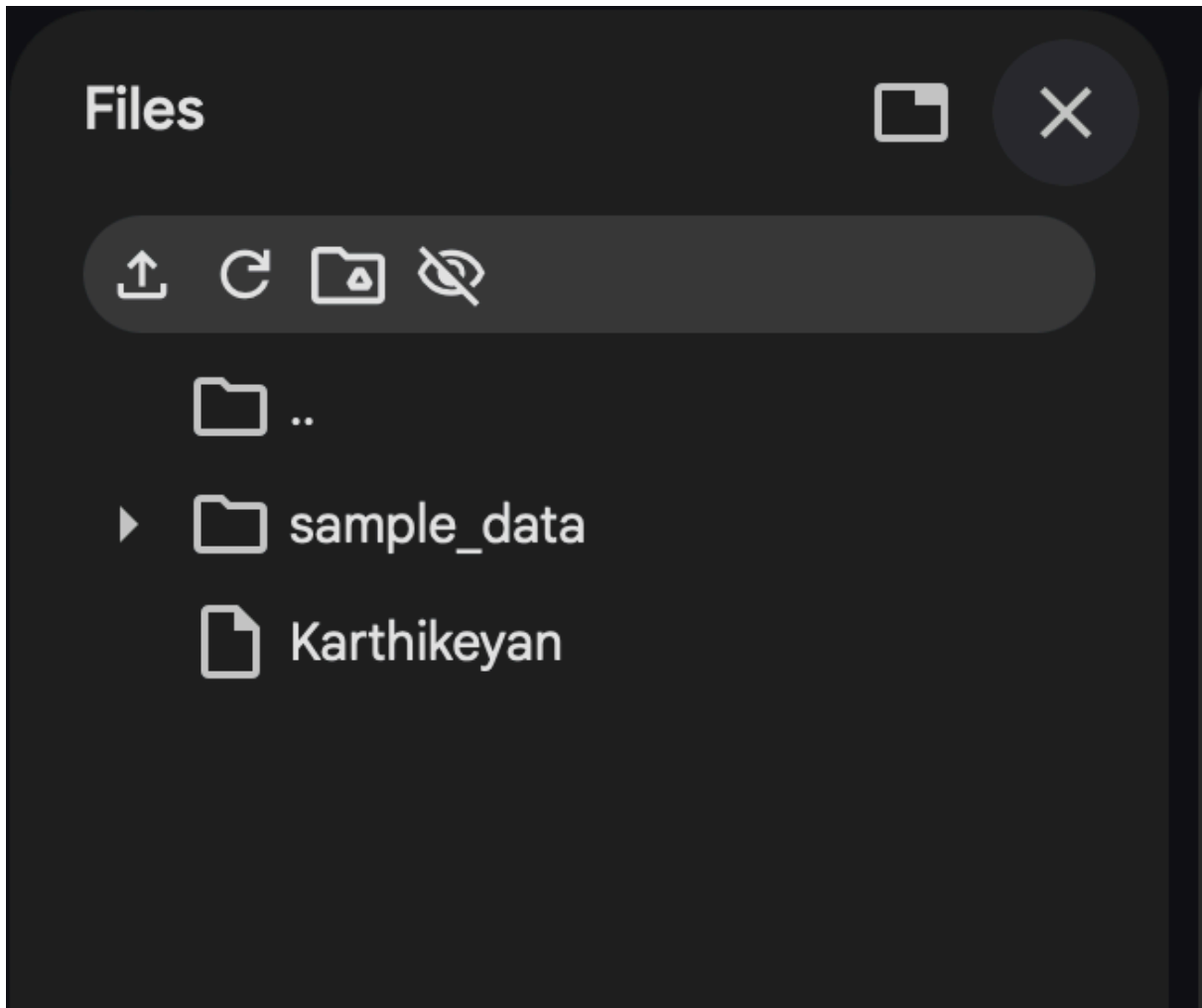
Exit with:

Ctrl + D

7. Create a Custom LLM (karthikeyan)

You created a Modelfile at:

/content/Karthikeyan



This file defines:

- Base model
- Parameters (temperature, context length)
- Strict system prompt
- Domain and scope restrictions

(Source configuration referenced from the uploaded Modelfile

Karthikeyan

)

Step 7.1 – Create the Custom Model

```
ollama create Karthikeyan -f /content/Karthikeyan
```

```

/content# ollama create Karthikeyan -f /content/Karthikeyan
[GIN] 2026/02/05 - 11:04:15 | 200 | 34.763µs | 127.0.0.1 | HEAD | "/"
[GIN] 2026/02/05 - 11:04:15 | 200 | 11.527346ms | 127.0.0.1 | POST | "/api/create"
gathering model components
using existing layer sha256:633fc5be925f9a484b61d6f9b9a78021eeb462100bd557309f01ba84cac26adf
using existing layer sha256:fa8235e5b48faca34e3ca98cf4f694ef08bd216d28b58071a1f85b1d50cb814d
using existing layer sha256:542b217f179c7825eeb5bca3c77d2b75ed05bafbd3451d9188891a60a85337c6
creating new layer sha256:f146cc3f87142b8495ae07152e890779c1e604837a5f2ce9a21b91b4147eb365
creating new layer sha256:4b731c7d3060c1af16578f50cac689a57adcc074468f8f84ca70d1cf88b15177
writing manifest
success
/content#

```

What This Does

- Uses phi3:mini as the base
- Applies your SYSTEM rules and parameters
- Produces a new model named karthikeyan

Step 7.2 – Verify Model Creation

ollama list

Expected output:

karthikeyan:latest

phi3:mini

8. Run the Custom Model

ollama run karthikeyan

```

bash: ollamalist: command not found
/content# ollama list
[GIN] 2026/02/05 - 11:04:51 | 200 | 33.337µs | 127.0.0.1 | HEAD | "/"
[GIN] 2026/02/05 - 11:04:51 | 200 | 476.499µs | 127.0.0.1 | GET | "/api/tags"
NAME ID SIZE MODIFIED
Karthikeyan:latest 1b22d8841288 2.2 GB 36 seconds ago
phi3:mini 4f2222927938 2.2 GB About a minute ago
/content# ollama run Karthikeyan:latest
[GIN] 2026/02/05 - 11:05:12 | 200 | 52.66µs | 127.0.0.1 | HEAD | "/"
[GIN] 2026/02/05 - 11:05:12 | 200 | 46.677692ms | 127.0.0.1 | POST | "/api/show"
[GIN] 2026/02/05 - 11:05:12 | 200 | 62.884011ms | 127.0.0.1 | POST | "/api/show"
time=2026-02-05T11:05:12.563Z level=INFO source=server.go:429 msg="starting runner" cmd="/usr/local/bin/ollama runner --olla
ma-engine --port 42769"
time=2026-02-05T11:05:12.928Z level=WARN source=cpu_linux.go:130 msg="failed to parse CPU allowed micro secs" error="strconv
.ParseInt: parsing \"max\": invalid syntax"
llama_model_loader: loaded meta data with 36 key-value pairs and 197 tensors from /root/.ollama/models/blobs/sha256-633fc5be
925f9a484b61d6f9b9a78021eeb462100bd557309f01ba84cac26adf (version GGUF V3 (latest))
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
llama_model_loader: - kv 0: general.architecture str = phi3
llama_model_loader: - kv 1: general.name str = model
llama_model_loader: - kv 2: general.type str = Phi 3 Mini 128k Instruct
llama_model_loader: - kv 3: general.finetune str = 128k-instruct
llama_model_loader: - kv 4: general.basename str = Phi-3
llama_model_loader: - kv 5: general.size_label str = mini
llama_model_loader: - kv 6: general.license str = mit
llama_model_loader: - kv 7: general.license.link str = https://huggingface.co/microsoft/P
hi-...
llama_model_loader: - kv 8: general.tags arr[str,3] = ["nlp", "code", "text-generation"]
llama_model_loader: - kv 9: general.languages arr[str,1] = ["en"]
llama_model_loader: - kv 10: phi3.context_length u32 = 131072
llama_model_loader: - kv 11: phi3.rope.scaling.original_context_length u32 = 4096
llama_model_loader: - kv 12: phi3.embedding_length u32 = 3072
llama_model_loader: - kv 13: phi3.feed forward length u32 = 8192

```

Behavior

- Responds ONLY to mobile security topics
- Rejects unrelated questions

- Enforces structured answers:
 - Title
 - Risks
 - Impact
 - Mitigation

9. Common Notes & Limitations

Colab Constraints

- CPU-only unless paid GPU
- Session resets will remove models
- Ollama must be reinstalled after runtime restart

Persistence Tip

- Save Modelfile in Google Drive
- Recreate model quickly using ollama create

Summary Flow

Colab Notebook

```
└─ XTerm
    └─ Install Ollama
    └─ Start ollama serve &
    └─ Pull base model
    └─ Create custom model
    └─ Run constrained LLM
```

Testing The Model with Question how it react

To verify Still the same model or Our customized model with some specific response only

❶ Basic Identity Check

Run:

```
ollama run Karthikeyan
```

2 Core Mobile Security Test (Must Pass)

Ask:

Analyze JWT storage risks in Android apps.

3 OWASP Mapping Test

Ask:

Analyze insecure data storage in Android apps and map to OWASP Mobile Top 10.

4 API Security Test

Ask:

Analyze mobile API token leakage risks.

5 Website / Product Analysis Test

Ask:

Analyze <https://gosense.ai> from a mobile security perspective.
Analyze <https://truecaller.com> from a mobile security perspective.

6 Unrelated Question (Refusal Test – CRITICAL)

Ask:

What is football world cup?

7 Hallucination Test

Ask:

Create a story about a hacker named John.

8 Stop-Control Test (Very Important)

Ask:

Analyze Android reverse engineering risks.

9 Consistency Test (Repeatability)

Ask the same question twice:

Analyze JWT storage risks in Android apps.

```
... >>> What is football world cup?
Title: FIFA World Cup
Risks: None related to my expertise in secure coding or architecture for Android and
iOS devices.
Impact: Not applicable within the scope of OWASP Mobile Top 10 vulnerabilities on
mobile applications.
Mitigation: N/A, as it does not pertain to securing a football world cup event from
cyber threats in terms of application security or data protection for users'
personal information and transactions during such events.[GIN] 2026/02/05 - 11:21:09 | 200 | 1.567305962s | 127.0.0.1 |
POST "/api/chat"

>>> Create a story about a hacker named John.
I focus exclusively on mobile security and vulnerability prevention.[GIN] 2026/02/05 - 11:21:50 | 200 | 349.595358ms |
127.0.0.1 | POST "/api/chat"

>>> Analyze Android reverse engineering risks.
Title: Risks of Reverse Engineering in the Context of Mobile Security for Android
Devices
Risks: Unauthorized access to proprietary algorithms, extraction of sensitive user
data through backdoors or vulnerabilities like buffer overflows and injection flaws.
Impact: Potential exposure of trade secrets leading to competitive disadvantage;
compromise of personal information resulting in identity theft risks for users.
Mitigation: Implementing robust encryption standards, conducting regular security
audits with tools like static analysis (e.g., FindBugs) and dynamic testing (e.g.,
OWASP ZAP), enforcing strict access controls based on least privilege principles,
keeping the Android OS updated to mitigate known vulnerabilities such as those
listed in CVE databases related to reverse engineering attempts.[GIN] 2026/02/05 - 11:22:10 | 200 | 2.6260703s | 127.
0.0.1 | POST "/api/chat"
```

```
... >>> Analyze https://gosense.ai from a mobile security perspective.
Title: Security Analysis of GSense AI for Mobile Devices
Risks: Potential data leakage through unsecured APIs, susceptibility to
man-in-the-middle (MITM) attacks if SSL pinning is not implemented correctly; risk
of malicious code injection via third-party libraries or plugins.
Impact: Loss of user trust due to privacy concerns and potential financial fraud
from compromised transactions facilitated by the AI's access to sensitive data like
payment information, banking details, etc.; reputational damage for GSense as a
security provider if vulnerabilities are exploited publicly.
Mitigation: Ensuring all APIs use strong authentication and encryption; implementing
certificate pinning techniques in app communications; conducting thorough code
reviews to identify potential injection points followed by sanitization of inputs,
employment of static application security testing (SAST) tools like SonarQube for
detecting vulnerabilities early on during the development lifecycle.[GIN] 2026/02/05 - 11:22:33 | 200 | 3.201634788s |
127.0.0.1 | POST "/api/chat"

>>> Analyze https://truecaller.com from a mobile security perspective.
Title: Security Analysis of TrueCaller Mobile Application
Risks: Risk of phishing attacks if users are misled into providing sensitive
information; potential for unauthorized access to user contacts and call logs
through exploitation of permissions or vulnerabilities in the app's code.
Impact: Exposure of personal communication patterns, leading to privacy violations;
risk of targeted scams using stolen contact lists if security is compromised.
Mitigation: Implementing multi-factor authentication for user accounts within
TrueCaller services; conducting regular penetration testing and code audits with
tools like Fortify or Checkmarx to identify potential vulnerabilities, ensuring that
permissions are granted strictly on a need-to-know basis (e.g., contact access only
when explicitly requested by the app), enforcing secure coding practices as
recommended in OWASP Mobile Top 10 guidelines such as input validation and output
encoding to prevent common injection attacks like SQL Injection or Cross-Site
Scripting (XSS).[GIN] 2026/02/05 - 11:24:37 | 200 | 3.545493619s | 127.0.0.1 | POST "/api/chat"
```

```

... >>> Analyze mobile API token leakage risks.
Title: Risks of Token Leakage in Mobile APIs for Android Devices
Risks: Unauthorized access to tokens through side channel attacks, cross-site request forgery (CSRF) vulnerabilities leading to unauthorized actions on behalf of the user; token interception during transmission if not properly secured with TLS. Impact: Compromise of API integrity and confidentiality resulting in data breaches or fraudulent transactions, undermining trust between users and service providers relying on these tokens for authentication purposes.
Mitigation: Employing token binding techniques to tie the access token with specific hardware (e.g., using platform-specific APIs like Android's AccountManager) ensuring that even if a token is intercepted, it cannot be used outside of its intended context; enforcing strict transport security policies by mandating HTTPS for all API communications and implementing certificate pinning to prevent MITM attacks; conducting regular code audits with tools such as SonarQube or Fortify SCA (Software Composition Analysis) to detect vulnerabilities that could lead to token leakage.[GIN] 2026/02/05 - 11:24:56 | 200 | 4.075365
298s | 127.0.0.1 | POST | "/api/chat"

>>> Analyze insecure data storage in Android apps and map to OWASP Mobile Top 10.
Title: Insecure Data Storage Practices in Android Applications Relating to the OWASP Mobile Top 10 Risks
Risks: Storing sensitive information like passwords or personal identifiers without encryption, leading to data exposure through physical extraction (e.g., rooting devices) or exploitation of storage vulnerabilities; improper handling and sanitization of input that could lead to injection attacks affecting stored data integrity.
Impact: Direct violation of user privacy with potential for identity theft, financial fraud if payment information is compromised; legal repercussions due to non-compliance with regulations like GDPR or CCPA which mandate secure handling and storage of personal data.
Mitigation: Adhering strictly to OWASP Mobile Top 10 guidelines by encrypting sensitive information using Android's Keystore system before storing it on the

```

```

Mitigation: Employ Android Keystore system with strong cryptographic algorithms (e.g., AES-GCM) for secure storage of JWTs, ensuring tokens are encrypted before being stored locally; implementing strict token validation on each API request to verify the integrity and expiration time using libraries like jjwt or java-jose in Android development environment; conducting regular security audits with tools such as OWASP ZAP for testing against vulnerabilities that could lead to JWT misuse, including but not limited to CSRF attacks which exploit improper token handling.[GIN] 2026/02/05 - 11:25:46 | 200 | 5.0822194
73s | 127.0.0.1 | POST | "/api/chat"

>>> What is football world cup?
Title: FIFA World Cup
Risks: None related to my expertise in mobile security and vulnerability prevention.
Impact: Not applicable within the scope of secure coding or architecture for Android devices, nor OWASP Mobile Top 10 vulnerabilities on mobile applications.
Mitigation: N/A as it does not pertain to securing a football world cup event from cyber threats in terms of application security or data protection during such events.[GIN] 2026/02/05 - 11:25:59 | 200 | 2.11
3216071s | 127.0.0.1 | POST | "/api/chat"

>>> Send a message (/? for help)

```